



آزمایشگاه طراحی سیستم های دیجیتال

آزمایش سوم-بخش اول
طراحی مقایسه کننده چهاربیتی

آرش فرح بخش - 402106286

سبحان زمانی - 402111574

امیر علی خدایی - 402105954

فهرست مطالب

هدف آزمایش	1
مقدمه	1
مقایسه کننده 1 بیتی و کد وریلاگ	Error! Bookmark not defined.
مقایسه کننده 4 بیتی و کد وریلاگ	Error! Bookmark not defined.
testbench	Error! Bookmark not defined.
خروجی های مدل سیم	Error! Bookmark not defined.
نتیجه گیری	9

هدف آزمایش

هدف از این آزمایش، طراحی یک مدار مقایسه کننده ۴ بیتی (4-bit Comparator) به صورت یک مدار ترکیبی است. مدار طراحی شده دو عدد دودویی ۴ بیتی به نام های A و B را دریافت کرده و با مقایسه آنها، سه خروجی تولید می کند:

• $A > B$ زمانی که A_gt_B

• $A = B$ زمانی که A_eq_B

• $A_{lt_B} < B$ زمانی که $A < B$

در این آزمایش، ابتدا یک مقایسه‌کننده ۱ بیتی طراحی شده و سپس با استفاده از چهار نمونه از آن، یک مقایسه‌کننده ۴ بیتی ساخته می‌شود. در نهایت عملکرد مدار توسط **Testbench** در محیط **ModelSim** بررسی می‌شود.

مقدمه

مقایسه‌کننده دیجیتال مداری ترکیبی است که برای مقایسه دو عدد دودویی استفاده می‌شود. خروجی این مدار مشخص می‌کند که کدام یک از دو عدد بزرگتر است یا اینکه آیا دو عدد با یکدیگر برابر هستند.

برای مقایسه دو عدد ۴ بیتی: $A = A_3A_2A_1A_0$ و $B = B_3B_2B_1B_0$

ابتدا بیت‌های با ارزش بالاتر یعنی A_3 و B_3 با یکدیگر مقایسه می‌شوند. اگر این دو بیت برابر باشند، بیت‌های بعدی یعنی A_2 و B_2 بررسی می‌شوند و این روند تا بیت کم‌ارزش ادامه پیدا می‌کند.

بنابراین بیت با ارزش بیشتر اولویت بالاتری در مقایسه دارد.

مقایسه‌کننده 1 بیتی و کد ورپلاگ

برای ساخت مدار ۴ بیتی، ابتدا یک مقایسه‌کننده ۱ بیتی طراحی می‌کنیم.

این مدار دو ورودی A و B و سه خروجی دارد.

جدول درستی مقایسه‌کننده ۱ بیتی به صورت زیر است:

A	B	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

معادلات منطقی خروجی‌ها عبارت‌اند از:

$$A > B = A\overline{B}$$

$$A = B = \overline{A \oplus B}$$

$$A < B = \overline{A}B$$

فایل زیر را با نام comparator_1bit.v ذخیره می‌کنیم:

```
module comparator_1bit (
    input A,
    input B,
    output A_gt_B,
    output A_eq_B,
    output A_lt_B
);

    assign A_gt_B = A & ~B;
    assign A_eq_B = ~(A ^ B);
    assign A_lt_B = ~A & B;

endmodule
```

در این کد از عملگرهای منطقی AND ، NOT و XOR برای پیاده‌سازی مدار مقایسه‌کننده استفاده شده است.

مقایسه‌کننده 4 بیتی و کد وریلاگ

فایل comparator_4bit.v:

```
module comparator_4bit (
    input [3:0] A,
    input [3:0] B,
    output A_gt_B,
    output A_eq_B,
    output A_lt_B
);
```

```
wire gt0, eq0, lt0;
```

```
wire gt1, eq1, lt1;
```

```
wire gt2, eq2, lt2;
```

```
wire gt3, eq3, lt3;
```

```
comparator_1bit c0 (
```

```
    .A(A[0]),
```

```
    .B(B[0]),
```

```
    .A_gt_B(gt0),
```

```
    .A_eq_B(eq0),
```

```
    .A_lt_B(lt0)
```

```
);
```

```
comparator_1bit c1 (
```

```
    .A(A[1]),
```

```
    .B(B[1]),
```

```
    .A_gt_B(gt1),
```

```
    .A_eq_B(eq1),
```

```
    .A_lt_B(lt1)
```

```
);
```

```
comparator_1bit c2 (
```

```
    .A(A[2]),
```

```
    .B(B[2]),
```

```
    .A_gt_B(gt2),
```

```
    .A_eq_B(eq2),
```

```
    .A_lt_B(lt2)
```

);

comparator_1bit c3 (

.A(A[3]),

.B(B[3]),

.A_gt_B(gt3),

.A_eq_B(eq3),

.A_lt_B(lt3)

);

assign A_gt_B =

gt3 |

(eq3 & gt2) |

(eq3 & eq2 & gt1) |

(eq3 & eq2 & eq1 & gt0);

assign A_lt_B =

lt3 |

(eq3 & lt2) |

(eq3 & eq2 & lt1) |

(eq3 & eq2 & eq1 & lt0);

assign A_eq_B =

eq3 & eq2 & eq1 & eq0;

endmodule

برای تعیین بزرگتر بودن A نسبت به B ، شرایط زیر در نظر گرفته می‌شود:

1. A3 از B3 بزرگتر باشد

2. در صورتی برابری شرط بالا A2 از B2 بزرگتر باشد

3. در صورت برابری دو شرط بالا A1 از B1 بزرگتر باشد

4. در نهایت A0 از B0 بزرگتر باشد

بنابراین رابطه منطقی خروجی A_gt_B به صورت زیر است:

$$A_gt_B = gt3 \text{ OR } (eq3 \text{ AND } gt2) \text{ OR } (eq3 \text{ AND } eq2 \text{ AND } gt1) \text{ OR } (eq3 \text{ AND } eq2 \text{ AND } eq1 \text{ AND } gt0)$$

به همین شکل، خروجی A_lt_B زمانی برابر 1 می‌شود که A از B کوچکتر باشد:

$$A_lt_B = lt3 \text{ OR } (eq3 \text{ AND } lt2) \text{ OR } (eq3 \text{ AND } eq2 \text{ AND } lt1) \text{ OR } (eq3 \text{ AND } eq2 \text{ AND } eq1 \text{ AND } lt0)$$

برای برابر بودن دو عدد نیز باید تمام چهار بیت با یکدیگر برابر باشند:

$$A_eq_B = eq3 \text{ AND } eq2 \text{ AND } eq1 \text{ AND } eq0$$

testbench

```
`timescale 1ns/1ps
```

```
module tb_comparator_4bit;
```

```
reg [3:0] A;
```

```
reg [3:0] B;
```

```
wire A_gt_B;
```

```
wire A_eq_B;
```

```
wire A_lt_B;
```

```
comparator_4bit uut (
```

```
    .A(A),
```

```
    .B(B),
```

```
    .A_gt_B(A_gt_B),
```

```
    .A_eq_B(A_eq_B),
```

```
    .A_lt_B(A_lt_B)
```

```
);
```

```
initial begin
```

```
    $monitor(
```

```
        "Time=%0t | A=%b | B=%b | A>B=%b | A=B=%b | A<B=%b",
```

```
        $time,
```

```
        A,
```

```
        B,
```

```
        A_gt_B,
```

```
        A_eq_B,
```

```
        A_lt_B
```

```
    );
```

```
A = 4'b0000;
```

```
B = 4'b0000;
```

```
#10;
```

```
A = 4'b0101;
```

```
B = 4'b0011;
```

```
#10;
```

```
A = 4'b0010;
```

```
B = 4'b0110;
```

```
#10;
```

```
A = 4'b1010;
```

```
B = 4'b1010;
```

```
#10;
```

```
A = 4'b1111;
```

```
B = 4'b0001;
```

```
#10;
```

```
A = 4'b0011;
```

```
B = 4'b0100;
```

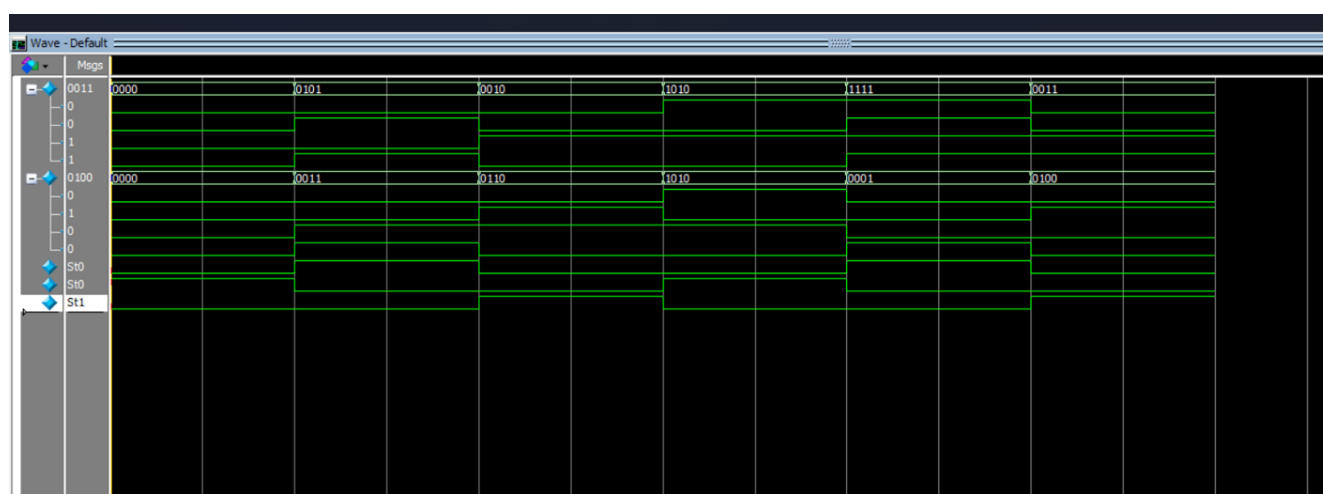
```
#10;
```

```
$finish;
```

```
end
```

```
endmodule
```

```
# Time=0 | A=0000 | B=0000 | A>B=0 | A=B=1 | A<B=0
# Time=10000 | A=0101 | B=0011 | A>B=1 | A=B=0 | A<B=0
# Time=20000 | A=0010 | B=0110 | A>B=0 | A=B=0 | A<B=1
# Time=30000 | A=1010 | B=1010 | A>B=0 | A=B=1 | A<B=0
# Time=40000 | A=1111 | B=0001 | A>B=1 | A=B=0 | A<B=0
# Time=50000 | A=0011 | B=0100 | A>B=0 | A=B=0 | A<B=1
# ** Note: $finish      : C:/Users/Amirali/Desktop/DSD_3/1/tb_compara
tor_4bit.v(56)
#      Time: 60 ns  Iteration: 0  Instance: /tb_comparator_4bit
```



نتیجه گیری

در این آزمایش یک مقایسه کننده ۴ بیتی با استفاده از زبان توصیف سخت افزار Verilog HDL طراحی و در محیط ModelSim شبیه سازی شد.

برای طراحی مدار ابتدا یک مقایسه کننده ۱ بیتی ایجاد شد و سپس با استفاده از چهار نمونه از این ماژول، مقایسه کننده ۴ بیتی ساخته شد. در این طراحی، بیت های با ارزش بالاتر اولویت بیشتری در مقایسه دارند.

با استفاده از Testbench، شش حالت مختلف شامل برابر بودن، بزرگتر بودن و کوچکتر بودن دو ورودی آزمایش شد. نتایج شبیه سازی نشان داد که خروجی های A_gt_B، A_eq_B و A_lt_B در تمام حالات مطابق انتظار عمل می کنند.

بنابراین می توان نتیجه گرفت که مدار طراحی شده یک مدار ترکیبی صحیح برای مقایسه دو عدد ۴ بیتی است و طراحی سلسله مراتبی آن با استفاده از چهار مقایسه کننده ۱ بیتی به درستی انجام شده است.